Expert Systems With Applications 186 (2021) 115736

Contents lists available at ScienceDirect

Expert Systems With Applications

journal homepage: www.elsevier.com/locate/eswa

Explaining anomalies detected by autoencoders using Shapley Additive Explanations

Liat Antwarg *, Ronnie Mindlin Miller, Bracha Shapira, Lior Rokach

The Department of Information and Software Systems Engineering, Ben-Gurion University of the Negev, Beer-Sheva, Israel

ARTICLE INFO

Keywords:
Explainable black-box models
XAI
Autoencoder
Shapley values
SHAP
Anomaly detection

ABSTRACT

Deep learning algorithms for anomaly detection, such as autoencoders, point out the outliers, saving experts the time-consuming task of examining normal cases in order to find anomalies. Most outlier detection algorithms output a score for each instance in the database. The top-k most intense outliers are returned to the user for further inspection; however, the manual validation of results becomes challenging without justification or additional clues. An explanation of why an instance is anomalous enables the experts to focus their investigation on the most important anomalies and may increase their trust in the algorithm. Recently, a game theory-based framework known as SHapley Additive exPlanations (SHAP) was shown to be effective in explaining various supervised learning models.

In this paper, we propose a method that uses Kernel SHAP to explain anomalies detected by an autoencoder, which is an unsupervised model. The proposed explanation method aims to provide a comprehensive explanation to the experts by focusing on the connection between the features with high reconstruction error and the features that are most important in terms of their affect on the reconstruction error. We propose a black-box explanation method, because it has the advantage of being able to explain any autoencoder without being aware of the exact architecture of the autoencoder model. The proposed explanation method extracts and visually depicts both features that contribute the most to the anomaly and those that offset it. An expert evaluation using real-world data demonstrates the usefulness of the proposed method in helping domain experts better understand the anomalies. Our evaluation of the explanation method, in which a "perfect" autoencoder is used as the ground truth, shows that the proposed method explains anomalies correctly, using the exact features, and evaluation on real-data demonstrates that (1) our explanation model, which uses SHAP, is more robust than the Local Interpretable Model-agnostic Explanations (LIME) method, and (2) the explanations our method provides are more effective at reducing the anomaly score than other methods.

1. Introduction

Recently, deep learning algorithms have been used for a wide variety of problems, including anomaly detection. While anomaly detection algorithms may be effective at identifying anomalies that might otherwise not be found, they have a major drawback, as their output is hard to explain. This shortcoming can make it challenging to convince experts to trust and adopt potentially beneficial anomaly detection systems. The output of such algorithms may contain anomalous instances that the domain expert was previously unaware of, and an explanation of why an instance is anomalous can increase the domain expert's trust in the algorithm. In addition, explanations can be contrastive, which is helpful and extremely relevant to explaining anomalies, since people do not usually ask why an event happened, but rather why one event happened instead of another event (Miller, 2019). The need to provide an explanation per instance (as opposed to providing an explanation for the whole model) was discussed in the 1970s in relation to expert systems (Shortliffe & Buchanan, 1975), but it has come to the fore again more recently, as models have become more complex. In some domains, such as the autonomous car and health domains, the lack of understanding and validating the decision process of a machine learning system is a disadvantage. The absence of a proper explanation makes it more difficult for decision-makers and domain experts to use the output of such algorithms. In addition to traditional performance measures such as accuracy and robustness, machine learning systems

The code (and data) in this article has been certified as Reproducible by Code Ocean: (https://codeocean.com/). More information on the Reproducibility Badge Initiative is available at https://www.elsevier.com/physical-sciences-and-engineering/computer-science/journals.

* Corresponding author.
E-mail addresses: liatant@post.bgu.ac.il (L. Antwarg), Ronniemi@post.bgu.ac.il (R.M. Miller), bshapira@bgu.ac.il (B. Shapira), liorrk@post.bgu.ac.il (L. Rokach).

https://doi.org/10.1016/j.eswa.2021.115736
Received 8 December 2020; Received in revised form 5 August 2021; Accepted 5 August 2021
Available online 11 August 2021
0957-4174/© 2021 Elsevier Ltd. All rights reserved.

should also consider ideas such as transparency (Olszewska, 2019). Transparency sheds light on how and why a machine learning system has made a particular decision. As black-box models are known for their lack of transparency (Adadi & Berrada, 2018), the need for reliable explanations arose. Such explanations build trust with users, help identify points of model failure, and remove barriers to entry for the deployment of deep neural networks in different domains. By building more transparent, explainable systems, users will better understand, and therefore trust, the intelligent agents (Kindermans et al., 2017; Miller, 2019; Ribeiro et al., 2016).

In the last decade, a few methods have been developed to explain predictions from supervised models. One approach uses an interpretable approximation of the original model (Lundberg & Lee, 2017); examples of methods that use an interpretable approximation of the original model include: LIME — Local Interpretable Model-agnostic Explanations (Ribeiro et al., 2016), which is an example of a model-agnostic method used to explain a prediction using a local model, and DeepLIFT (Shrikumar et al., 2017), which is an example of a model-specific method for explaining deep learning models in which the contributions of all neurons in the network are backpropagated to the input features. SHAP — SHapley Additive exPlanation (Lundberg & Lee, 2017) combines previous methods for explaining predictions by calculating feature importance, using Shapley values from game theory which ensure consistency of the explanations.

Autoencoders have become widely adopted for unsupervised anomaly detection tasks (Erfani et al., 2016; Paula et al., 2016; Sakurada & Yairi, 2014). An autoencoder is an unsupervised algorithm that represents the normal data in lower dimensionality and then reconstructs the data in the original dimensionality; thus, the normal instances are reconstructed properly, and the outliers are not, making the anomalies clear. To the best of our knowledge, no previous research has been performed that uses SHAP to provide black-box explanations for anomalies revealed by an autoencoder. In this paper, we present a new method based on SHAP values, to explain the anomalies found in an autoencoder's output. The method could be beneficial to experts requiring justification and visualization regarding anomalies. Domain experts that used the explanations on real-world data provided positive feedback, claiming that the explanations helped them understand and investigate the anomalies. In addition, to evaluate the proposed method, we created autoencoders for which we know the connections between the features; we then used an artificial dataset to examine whether our method uses the correct set of features to explain the anomalies. We also evaluated the robustness of our method using 'noise' features and investigated how the explanations can be used to reduce the anomaly score of an instance.

2. Background

2.1. Explaining black-box models

Explainable AI, also known as explainable artificial intelligence, XAI or interpretability, is a research field related to trust and transparency of AI-based systems (Adadi & Berrada, 2018; Arrieta et al., 2020; Gilpin et al., 2018; Olszewska, 2019). Doshi-Velez and Kim (2017) defined interpretability as the ability to explain or present in terms understandable to a human. Montavon et al. (2017) defined interpretability as the ability of a system to map an abstract concept into a domain that humans can make sense of, like images or texts. They defined explainability as the ability of a system to produce a collection of features of the interpretable domain that contributed to producing a decision for a given example. In contrast, some studies refer to both terms (explainability and interpretability) interchangeably. Miller (2019) describes both terms as generating decisions in which one of the criteria taken into account during the computation is how well a human could understand the decisions in the given context. In this paper we use the term explainability and aim to provide a set of features that contributed to an anomalous instance's reconstruction error.

The need to provide an explanation per instance (as opposed to providing an explanation for the whole model) came to the fore fairly recently, as models become more complex. Since 2018, this need became more urgent due to the General Data Protection Regulation (GDPR) that requires algorithms that make decisions based on user-level predictors, which "significantly affect" users, provide an explanation. Individuals affected by algorithmic decisions have a *right to an explanation* (Doshi-Velez & Kim, 2017; Goodman & Flaxman, 2017), and this right is enabled by explaining the logic of black-box models (Guidotti et al., 2018). Lipton (2018) define transparency as the opposite of black-boxness. According to Adadi and Berrada (2018), true transparency is provided by using interpretable models, such as decision trees, rules, or linear models, while non-transparent black-box models require techniques for post-hoc explanations (Adadi & Berrada, 2018). When a post-hoc explanation for a decision is required, there is a need for reasons or justifications for a specific instance, and not an explanation of how a model functions in general (Adadi & Berrada, 2018; Mittelstadt et al., 2019). The quality of the explanation is highly related to the quality of the model. A good model will recognize the domain concepts, and thus the quality of the explanation should be better (Štrumbelj et al., 2009).

Reliable explanations build trust with users, help identify points of model failure, and remove barriers to the deployment of deep neural networks in different domains. By building more transparent, explainable systems, users will better understand, and therefore trust, the intelligent agents (Ribeiro et al., 2016, Miller, 2019, Kindermans et al., 2017). Some models already have a transparent decision-making process from which it is easy to extract an explanation. For example, a decision from decision trees for an instance can be explained by observing the rules that lead from the root to the leaf that contains the instance. For most other, less transparent models, model-dependent and model-agnostic approaches have been developed (Štrumbelj et al., 2009).

2.2. Anomaly detection

Anomaly detection, also referred to as outlier detection (Hodge & Austin, 2004) is the problem of finding patterns in data that do not conform to expected behavior (Chandola et al., 2009). While outliers are usually considered as an error or noise, they may carry important insights on the system (Ben-Gal, 2005). In some applications, it can be useful to look at the outliers in the dataset rather than finding patterns in the data (Breunig et al., 2000). Many applications can benefit from applying outlier detection methods. These methods can be implemented for intrusion detection in cyber security; fraud detection for credit cards, insurance, or healthcare; fault detection in critical systems (Hodge & Austin, 2004; Singh & Upadhyaya, 2012); and instance selection for reducing the dataset size and improving the machine learning model and its runtime (Carbonera & Olszewska, 2019; Liu & Motoda, 2013; Olvera-López et al., 2010).

Another definition for anomaly is "an observation which deviates so much from the other observations as to arouse suspicion that it was generated by a different mechanism" (Aggarwal, 2015). Therefore, the importance of anomaly detection is the fact that finding anomalies can provide significant actionable information that can be critical to the system (Chandola et al., 2009).

Anomaly detection algorithms can be divided into three main categories (Aggarwal, 2015; Bergman & Hoshen, 2020; Takeishi & Kawahara, 2020). The first is **distribution-based** methods that rely on the nature of anomalous records to have a lower probability of appearing in any anomaly detection dataset. Such methods aim to model the distribution of the normal samples. The second category is **reconstruction-based** methods which rely on the idea of reconstructing samples. While normal samples can be reconstructed successfully using a set of

functions, the reconstruction is not trivial for anomalous data and the reconstruction error of an instance is used as an anomaly score. Data points with a high reconstruction error are defined as anomalies (An & Cho, 2015). In recent years, the use of autoencoder-like models has become popular (Zhou & Paffenroth, 2017). The third category is **classification-based** methods, which aim to separate the area that contains normal records from other areas by learning a good feature space. Recently, deep learning methods based on classification have shown promising results (Chen et al., 2017; Erfani et al., 2016; Golan & El-Yaniv, 2018; Hawkins et al., 2002; Song et al., 2017).

Based on the accessibility of labels and characteristics of the training data, anomaly detection techniques can either be supervised, semi-supervised, or unsupervised. Collecting large-scale labeled anomalous data, as required for the supervised techniques, can be time-consuming and expensive, as it requires the manual work of a domain expert. As a result, fully supervised anomaly detection is often impractical and even infeasible (Pang et al., 2020). To avoid this issue, recent studies have focused on the widely applicable unsupervised anomaly detection techniques. These techniques lack any prior knowledge of true anomalies, but they make an implicit assumption that normal instances are significantly more frequent than anomalies in the test data and rely heavily on this assumption.

One such technique is an autoencoder. Autoencoders were first introduced in the 1980s (Rumelhart et al., 1985), and in the last decade they have been widely used in deep architectures (Bengio et al., 2007; Hinton et al., 2006; Hinton & Salakhutdinov, 2006). An autoencoder is an unsupervised neural network that is trained to produce target values equal to its inputs (Goodfellow et al., 2016). An autoencoder represents the data in lower dimensionality (encoding) and reconstructs the data in the original dimensionality (decoding). Based on the input, the autoencoder learns an identity function so that the autoencoder's output is similar to the input, and the embedded model created in encoding represents normal instances well. In contrast, anomalies are not reconstructed well and have a high amount of reconstruction error, so in the process of encoding and decoding the instances, the anomalies are discovered.

2.3. SHAP

A few methods have been developed to explain predictions from supervised models. The SHAP (SHapley Additive exPlanation) framework (Lundberg & Lee, 2017) gathers previously proposed explanation methods, such as LIME (Ribeiro et al., 2016) and DeepLIFT (Shrikumar et al., 2017), under the class of additive feature attribution methods. Methods in that class are explanation models in the form of a linear function of simplified binary variables, as in $f(x) = g(z) = \theta_0 + \sum_{i=1}^{m} \theta_i z_i$, where $f(x)$ is the original model (an autoencoder in this paper); $g(x)$ is the explanation model; z is the simplified input; $x = h_x(z)$ is a mapping function to the original method; and $\theta_0 = f(h_x(0))$ is the model output without all of the simplified inputs.

SHAP has a sound theoretic basis, which is a benefit in regulated scenarios. It uses Shapley values from game theory to explain a specific prediction by assigning an importance value (SHAP value) to each feature that has the following properties: (1) local accuracy — the explanation model has to at least match the output of the original model; (2) missingness — features missing in the original input must have no impact; (3) consistency — if we revise a model such that it depends more on a certain feature, then the importance of that feature should not decrease, regardless of other features.

Lundberg and Lee (2017) demonstrate that SHAP is better aligned with human intuition than previous methods, since it has those properties.

The SHAP framework suggests a model-agnostic approximation for SHAP values, called Kernel SHAP. Kernel SHAP uses linear LIME (Ribeiro et al., 2016), along with Shapley values, to build a local explanation model. The local explanation model is a weighted linear regression which is built on a background set and a sample from the possible coalitions of features in the data. We use Kernel SHAP as it provides more accurate estimates with fewer evaluations of the original model than other sampling-based estimates.

3. Related work

3.1. Explaining anomalies

According to Amarasinghe et al. (2018), the ability to explain an anomaly detection model in critical areas, such as infrastructure security, is almost as important as the model's prediction accuracy. Thus, effective anomaly explanation would significantly improve the usability of anomaly detection techniques in real-world applications. One of the motivations for explaining anomalies revealed by an anomaly detection algorithm is to bridge the gap between detecting and identifying domain-specific anomalies. In other words, some of the outliers are actually interesting anomalies, and others are revealed by the algorithm as outliers only because they are rare (and not interesting as anomalies). Providing reasons for outlierness can significantly reduce the manual inspection performed by domain experts (Liu et al., 2018).

3.1.1. Explaining supervised anomalies

Most papers about anomalies' explanations deal with anomalies revealed by a supervised model. Among them, Liu et al. (2018) proposed the Contextual Outlier INterpretation (COIN) framework to explain anomalies spotted by detectors, using important features, the degree of abnormality, and the contrastive context of the anomaly. Goodall et al. (2019) presented "Situ", a streaming anomaly detection and visualization system for discovering and explaining suspicious behavior in computer network traffic and logs. The explanation is based on visualizing the context of the anomaly. Collaris et al. (2018) designed two dashboards that provide explanations for fraud detected by a random forest algorithm. The explanations are based on existing explanation algorithms, such as the instance-level feature importance method (Palczewska et al., 2014), partial dependence plots (Friedman, 2001), and local rule extraction (a variation of LIME Ribeiro et al., 2016). Arp et al. (2014) presented a method for the detection of Android malware using SVM and explain the decisions made by identifying the features that most contribute to a detection and checking whether the extracted features which contribute to the detection match common malware characteristics. A recent study suggested an approach to explain an anomaly revealed by any anomaly detector by creating a random forest with only that anomaly in the anomalous class, and normal cases in the other class. Then, they extracted rules that explain the classification of that anomaly (Kopp et al., 2020).

3.1.2. Explaining unsupervised anomalies

There are not many works about explaining the results of unsupervised models. The following recent studies are related to ours, relevant both for explaining anomalies and unsupervised models. One of them compared Shapley values to reconstruction errors of features in principal component analysis (PCA) for explaining anomalies (Takeishi, 2019). In their research the authors changed a feature to make the instance anomalous and then compared the Shapley values to the reconstruction errors, while in our research, we used features' Shapley values to explain features with a high reconstruction error. Another study explains network anomalies revealed by variational autoencoders (Nguyen et al., 2019). Unlike our method, which treats the autoencoder as a black-box and provides explanations for the anomalies, they explained the anomalies by analyzing the gradients, in order to identify the main features that affect the anomaly. We chose to work with a black-box explanation model, because it has the advantage of being able to explain any autoencoder, without knowledge of the exact architecture of the model; in our case, just the input and output are known.

Although not related specifically to anomalies, we chose to mention some works about explaining clustering, since clustering algorithms are widely used algorithms for unsupervised learning. There has been limited success in addressing the issue of cluster interpretability (Bertsimas et al., 2018). Studies have aimed at explaining clustering in different ways, such as the features in the centroid's center, visualization using PCA or t-SNE, or by building decision trees from the clusters (Bertsimas & Dunn, 2017; Jolliffe, 2011; Kauffmann et al., 2020; Liu et al., 2000; Lundberg et al., 2018; Maaten & Hinton, 2008; Radev et al., 2004).

3.2. Explanation evaluation

Although the number of explainability methods and papers has increased in recent years, there is little consensus on the exact definition of explainability or interpretability in the context of machine learning and how to evaluate it. Adadi and Berrada (2018) believe that the subjective nature of explainability is the reason for this. Doshi-Velez and Kim (2017) suggested three types of evaluations: (1) Application-grounded — real humans, real tasks, (2) Human-grounded — real humans, simplified tasks, and (3) Functionally-grounded — no humans, proxy tasks. Although it would be ideal to evaluate our explanations using domain experts who could assess the effectiveness of the explanation for them, such experts are not always available and do not always know specifically what they want to investigate (in terms of which instances may be anomalous and where to start their investigation of a potentially anomalous instance) before using a prototype (Liu et al., 2017).

Some studies suggested categories to evaluate the explanations. Gunning (2017) divided explanation effectiveness into five categories: mental model, task performance, trust assessment, correctability, and user satisfaction. Hoffman et al. (2018) defined key concepts for measuring explanation of an AI system: the goodness of explanations, user satisfaction, users' understanding of the AI system, the effect of curiosity on the search for explanations, user trust, and performance. Melis and Jaakkola (2018) used Explicitness, Faithfulness and Stability for evaluation.

Lundberg et al. (2018) evaluated their explanation algorithm by creating two simple decision trees, so they knew which features contributed to a prediction. Yang and Kim (2019) recently presented a benchmark for prediction explanation evaluation called BIM (benchmark interpretability method). To construct this benchmark, they added noise objects to images and trained a classifier, without expecting that the noise objects would play a role in explaining the classification. There are a few studies (Lundberg et al., 2018; Lundberg & Lee, 2017; Samek et al., 2017; Shrikumar et al., 2017) that evaluated prediction explanations by changing the features that explained the most predicted class, with the goal of reducing the probability for the predicted class or even changing the predicted class. For evaluating the explanation model in this paper, we used ideas from these papers.

4. Explaining autoencoder anomalies

Our challenge was to design a method for explaining an anomaly revealed by an autoencoder (unsupervised), unlike existing explainability methods which are used to explain a prediction (supervised). We use an autoencoder to detect anomalies through the reconstruction error (anomaly score). Instances with a high anomaly score are considered anomalous. We explain an anomaly score, which is the difference (error) between the input value and the output (reconstructed) value. An anomaly, if it exists, resides in the values of the input, and the explanatory model needs to explain why this instance was not predicted (reconstructed) well. Thus, an explanation must be connected to the error. Our method therefore computes the SHAP values of the reconstructed features and relates them to the true (anomalous) values in the input.

Given input instance X with a set of features $x_1, x_2, \ldots, x_n$ and its corresponding output X' and reconstructed values $x'_1, x'_2, \ldots, x'_n$, using an autoencoder model f, the reconstruction error of the instance is the sum of errors of each feature $L(X, X') = \sum_{i=1}^{n}(x_i - x'_i)^2$. Let $x_{(1)}, \ldots, x_{(n)}$ be a reordering of the features in $errorList$, such that $|x_{(1)} - x'_{(1)}| \geq \cdots \geq |x_{(n)} - x'_{(n)}|$, $topMfeatures = x_{(1)}, \ldots, x_{(m)}$ contains a set of features for which the total corresponding errors $topMerrors$: $|x_{(1)} - x'_{(1)}|, \ldots, |x_{(m)} - x'_{(m)}|$ represent an adjustable percent of $L(X, X')$. Our method explains, by using SHAP values, which features affected each of the high reconstruction errors in $topMfeatures$.

Algorithm 1 Calculate SHAP values for $topMfeatures$

Input: X — An instance we want to explain, $ErrorList$ — An ordered list of error per feature, $X1..j$ — Instances that Kernel SHAP uses as background examples, i — The feature we explain in X, $|s|$ — The number of coalitions of features to approximate Shapley values, f — autoencoder model
Output: $shaptopMfeatures$ — SHAP values for each feature within $topMfeatures$

1: $topMfeatures \leftarrow$ top values from $ErrorList$
2: **for all** $i \in topMfeatures$ **do**
3: $\quad explainer \leftarrow shap.KernelExplainer(f, X1..j)$
4: $\quad shaptopMfeatures[i] \leftarrow explainer.shapvalues(X, i, |s|)$
5: return $shaptopMfeatures$

Algorithm 1 presents the pseudocode for the process. First, we extract the features with the highest reconstruction error from the $ErrorList$ and save them in the $topMfeatures$ list. Next, for each feature x'_i in $topMfeatures$, we use Kernel SHAP to obtain the SHAP values, i.e., the importance of each feature $x_1, x_2, \ldots, x_n$ (except for x_i) in predicting the examined feature x'_i. Kernel SHAP receives f and a background set with j instances to build the local explanation model and calculate the SHAP values, along with $|s|$, the number of coalitions of features to approximate Shapley values. Then, f takes X and i as input and predicts X'; the value in the i'th feature (a feature in the $topMfeatures$) is returned by Algorithm . The result of this step is a two-dimensional list, $shaptopMfeatures$, in which each row represents the SHAP values for one feature from the $topMfeatures$.

Algorithm 2 f(X,i) - predict the i'th feature

Input: X - An instance we want to explain, i - The feature for which we want a prediction (from the output vector of the autoencoder), f - Autoencoder model
Output: x'_i - The value of the i'th feature in X'

1: $x'_i \leftarrow f.predict(\mathrm{X})[i]$
2: return x'_i

We divide the SHAP values into values *contributing* to the anomaly — those pushing the predicted (reconstructed) value away from the true value, and values *offsetting* the anomaly — those pushing the predicted value towards the true value. The pseudocode in Algorithm 3 presents how the SHAP values are divided. For each feature (line 1), we check if the true (input) feature value is greater than the predicted value (line 2); the *contributing* features are the features with a negative SHAP value (line 3), and the *offsetting* features are the positives (line 4). If the predicted feature value is greater than the actual (input) value (line 5), then the *contributing* features are the features with a positive SHAP value, and the *offsetting* features are the negatives. This algorithm returns two lists, $shapContributing$ and $shapOffsetting$, that contain the *contributing* and *offsetting* features, along with their SHAP values, for each of the $topMfeatures$.

Algorithm 3 Divide SHAP values into those contributing and offsetting an anomaly

Input: $shaptopMfeatures$ - SHAP values for each feature, X - An instance we want to explain, X' - The prediction for X
Output: $shapContribute$, $shapoffset$

for all $i \in shaptopMfeatures$ **do**
  $shapNegatives \leftarrow$ features with negative SHAP values from $shaptopMfeatures[i]$
  $shapPositives$ features with positive SHAP values from $shaptopMfeatures[i]$
  if $x_i > x'_i$ **then**
    $shapContribute[i] \leftarrow shapNegatives$
    $shapOffset[i] \leftarrow shapPositives$
  else
    $shapContribute[i] \leftarrow shapPositives$
    $shapOffset[i] \leftarrow shapNegatives$
return $shapContribute$, $shapOffset$

The next step is selecting the features with high SHAP values for each feature in the list $topMfeatures$; so from each row in $shapContributing$ and $shapOffsetting$, we extract the highest values. Since our goal is to help the domain expert understand why an instance is an anomaly, we present the explanation in the form of a table that depicts the *contributing* and *offsetting* anomaly features, using colors which correspond to the SHAP values (Fig. 4(b)). A higher SHAP value (depicted by a darker color) means that the feature is more important for the prediction (*contributing* features appear in red and *offsetting* features appear in blue). The flow chart describing the process of providing an explanation for an anomaly revealed by an autoencoder can be seen in Fig. 1.

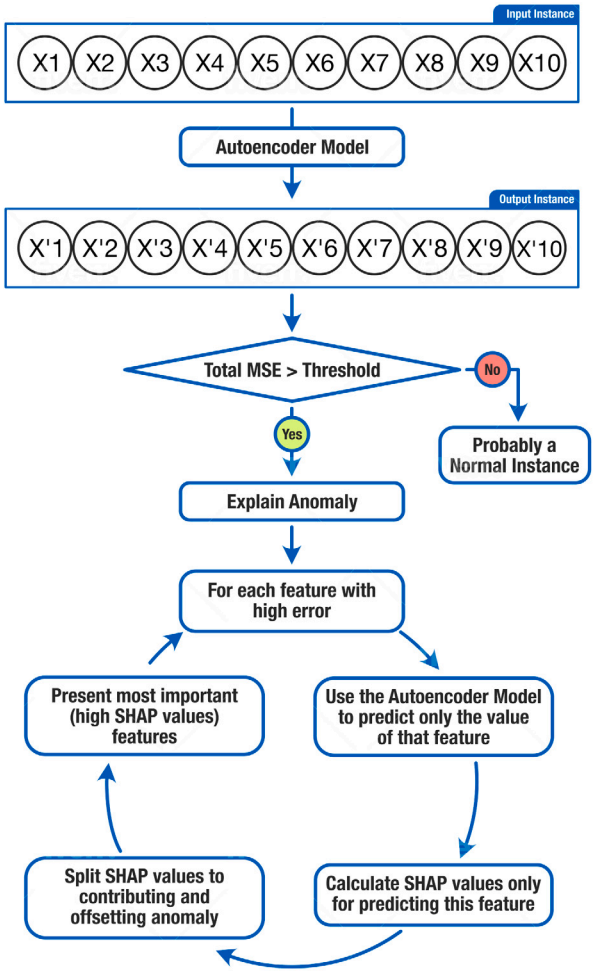


Fig. 1. Flow chart describing the process of providing an explanation for an anomaly revealed by an autoencoder.

5. Example

To demonstrate our method, we use an example in which we are trying to detect drug abuse using a prescription database. Each record has 10 features that may point to drug abuse. The instance presented in Fig. 2(a), which has a high reconstruction error, is a prescription for a large amount of painkillers prescribed to a 30-years-old man who has no comorbidities but was recently involved in a car accident.

Extracting top error features. Since the total reconstruction error is calculated from the error of each feature ($|X_i - X'_i|$), we can extract the features with the highest reconstruction error. Let us assume that features X_3 (drug amount), X_6 (days between prescription and purchase date), and X_8 (doctor's name) have the highest reconstruction errors; therefore, these are the features that we explain using SHAP.

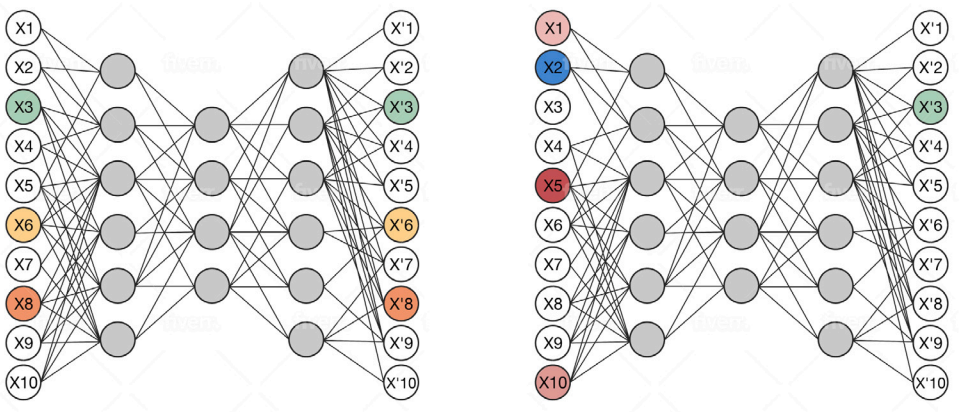
Calculating SHAP values for a feature with high reconstruction error. In order to explain the reconstruction error in the drug amount feature (X_3), we use the autoencoder to predict the value of the drug amount X'_3, as in Fig. 2(b), and use SHAP to obtain the importance of each feature in the network in predicting X'_3, relative to a baseline which is calculated using the background set, as in Algorithm 1.

Features contributing or offsetting an anomaly. Fig. 4(a) presents a plot with positive (depicted in blue in the figure) and negative (red) SHAP values. Assume that the real value of feature X_3 is one, and the autoencoder predicted that X'_3 equals 0.01. To divide the features based on whether they *contribute* or *offset* the anomaly, we use the true (input) value, output (reconstructed) value, and the polarity of the SHAP values, as in Algorithm 3. Only event=car accident (X_2) pushed the value towards the true value, *offsetting* the anomaly, while time from last prescription=five days (X_5), age=30 (X_{10}), and medical background=no disease (X_1) pushed the value away from the true value towards the prediction, *contributing* to the anomaly. Because the young patient had no comorbidities and requested painkillers five days before, the autoencoder predicted that the amount should be much lower than what was prescribed. Perhaps the event feature (X_2) offsets the anomaly, because the fact that the patient was involved in a car accident makes this prescription correct.

Depiction of contributing and offsetting anomaly features. Fig. 4(b) shows how we visually depict the features *contributing* and *offsetting* the anomaly for the domain expert. For each feature in the $topMfeatures$ (X'_3, X'_6, X'_8), we show the *contributing* features in red. For example, X_5 is the feature that contributed most to the error of feature X_3. Then we present the features that offset the anomaly in blue; the last column contains the real value of that feature.

6. The motivation for using our method for explaining anomalies

The prescription described in the example in Section 5 may be normal. Painkillers are commonly prescribed, even for young, healthy people. So why is it anomalous? Without using our suggested method to provide an explanation for the anomalies revealed by the autoencoder, the domain expert would receive an alert regarding this prescription, and the only clue he/she would have about its anomalous nature is the list of features with the highest reconstruction errors. In this example the clues would be the features of drug amount, days between prescription and purchase date, and doctor's name. Without the explanations of the features that are most important in affecting the reconstruction error, the reason for the anomaly remains vague. Using our method, we



(a) Drug amount (X_3), days between prescription and purchase date (X_6), and doctor's name (X_8) are the features with the highest errors

(b) Calculating contributing and offsetting features for drug amount

Fig. 2. Explaining the drug amount (X_3) error.

are able to explain what affected the incorrect predictions (and thus the reconstruction errors).

When the expert sees the features that most affected the prediction (the features that explain the reconstruction error), it is clearer why the autoencoder detected it as an anomaly. In this example we had only 10 features, but in many real-world problems, the number of features is much higher, which makes it much harder to understand the anomaly without a proper explanation.

Another way of using SHAP to explain anomalies is to add another layer to the autoencoder after it is trained, in such a way that each neuron represents the reconstruction error of one feature, and then add an output layer that sums the reconstruction errors of all of the features, as in Fig. 3.

In this case we change the explanation task to a regression task and use Kernel SHAP without adaptations, in order to obtain the features that are the most important in affecting the total reconstruction error of the instance, rather than obtaining the features that are most important in a prediction for each feature with high reconstruction error separately. We conducted tests using this approach and found that in most of the cases, the features with the highest SHAP values were the same as the features with the highest reconstruction errors. These findings strengthened our confidence in our decision to use SHAP for explanations and based on this positive finding, we continued with our method, because it provides more comprehensive explanations to experts by focusing on the connection between the features with high reconstruction error and the features that are most important in affecting the reconstruction error.

7. Evaluation

We evaluated our suggested method for explaining anomalies using four different approaches: (1) we used simulated data in which we know which features should explain the anomalies, (2) we assessed the robustness of the explanations on real-world data, (3) we examined the effect of changing the value of the features that explain the anomaly on the anomaly score, and (4) we interviewed domain experts that used the explanations on real data.

7.1. Datasets

We performed the evaluations on the four datasets described below. Different evaluations were performed on different datasets.

7.1.1. Warranty claims

This unsupervised dataset contains 15,000 warranty claims from a large car manufacturer, with 1,000 features per instance. To detect anomalies, we trained an autoencoder and provided a list of 114 claims with high anomaly scores to domain experts for inspection. The threshold for choosing the top anomalies was determined using the interquartile range (IQR) measure. The top anomalies are the ones for which $Reconstructionerror > Q3 + (Q3 - Q1) * 1.5$). After the domain experts examined the top anomalies, we received feedback regarding which of the claims were in fact interesting anomalies. For privacy reasons, this dataset is not public, and we used it in our study only for the experts' evaluation of the proposed explanation method.

7.1.2. KDD Cup 1999

The KDD Cup 1999 dataset (External Data Source, 2018b), obtained from the UCI Machine Learning Archive, is a well-known anomaly detection dataset of intrusions simulated in a military network environment. It has been widely used in many studies. We use the '10%' version of the data which is a subset of the data that contains records classified as 22 different types of attacks. For the evaluation of our method, we ignored 21 of these types of attacks and used only the most frequent attack after the normal class. Specifically, we used the "back" attack which is a type of denial-of-service attack, resulting in a binary dataset containing 97,278 normal connections (class zero) and 2,203 'back' attack connections (class one). This dataset contains 41 features.

7.1.3. Credit Card Fraud Detection

The Credit Card Fraud Detection dataset (External Data Source, 2018a) contains credit card transactions from two days in September 2013 for European cardholders; in this dataset, there are 492 fraudulent transactions out of a total of 284,807 transactions. This data contains 30 numerical features which are the result of a preprocess PCA transformation. Due to confidentiality issues, we have no information about the meaning of the features, but the given features, V1, V2, V3 ... V28, are principal components obtained with PCA. 'Time' and 'Amount' are the only features that have not been transformed with PCA. The feature 'Class' is the label — one in case of fraud and zero otherwise.

7.1.4. Artificial dataset

We generated an artificial dataset that consists of one million instances with six features, using the following procedure: The first four

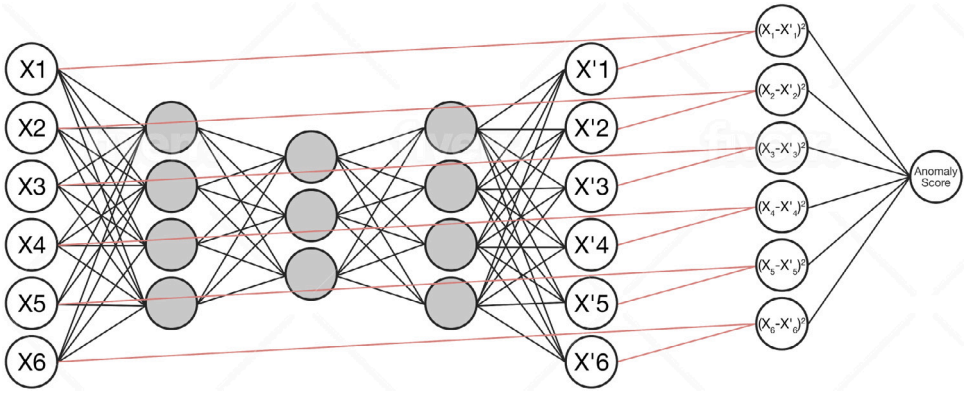
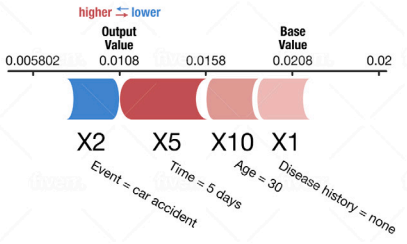


Fig. 3. Adding a layer of the total reconstruction error to provide explanations for that error using SHAP.



(a) Explaining X'_3

Top Errors List	Contribution to Anomaly	Offsetting Anomaly
X'3	X5	X2
real 1.0	X10	
predicted 0.01	X1	
X'6	X7	X5
real 0	X2	X1
predicted 0.8		X3
X'8	X5	X7
real 0.84	X1	X3
predicted 0.45	X2	X6

(b) Features that contribute and offset the anomaly

Fig. 4. (a) Highest SHAP values in explaining drug amount (X'_3). The real value of X_3 is one, so features X_5, X_{10}, and X_1 push the value of X'_3 away from the true value, *contributing* to the anomaly. Feature X_2 pushes the value towards the true value of X_3, *offsetting* the anomaly. (b) A table containing explanations for all features with high reconstruction error.

features, X_1, X_2, X_3 and X_4, were generated randomly with feature values between zero and one.

The following two features are a linear combination of some of the four previous features: $X_5 = X_1+X_2$, $X_6 = X_3+X_4$. Each such linear combination yields two other linear combinations of the features: $X_1 = X_5 - X_2$ and $X_2 = X_5 - X_1$. The same applies to features X_3, X_4, and X_6. Then, we randomly selected 5,000 records to create anomalies by changing X_5 or X_6 to a random value between zero and one, as seen in Table 1.

7.2. Correctness of explanations

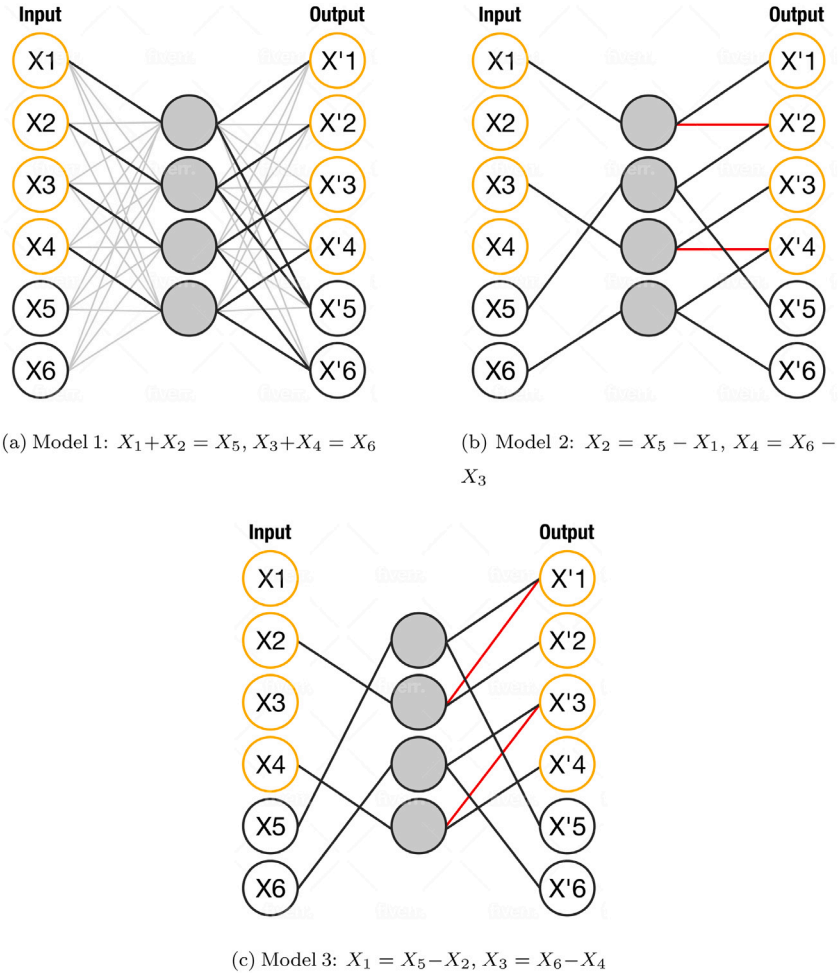
To evaluate the correctness of our method of explaining anomalies detected by an autoencoder using SHAP, we built "perfect" autoencoders for which we know the relations between the features and thus have a ground truth to explain the anomaly. The effectiveness of our method is demonstrated when the anomalies are explained using the known correct set of features. For this evaluation we did not split the features into those that contribute and offset the anomaly, since we were interested in examining all of the explanatory features. This approach is similar to the evaluation approach used in Lundberg et al. (2018), where they created two simple decision trees so they would know which features are attributed to a prediction. We used the same idea and built autoencoders with known connections, so that when there is an anomaly, we would know which features explain it. The dataset used for this evaluation is artificial, as explained in Section 7.1.4.

Table 1
Nine examples of the artificial dataset and anomalies (generated anomalies in bold): class zero — normal cases, class one — generated anomalies (the value of X_5 or X_6 does not fit the linear combination).

X_1	X_2	X_3	X_4	X_5	X_6	Class
0.447	0.608	0.445	0.869	**0.34**	1.314	1
0.499	0.481	0.386	0.862	0.98	1.248	0
0.05	0.287	0.275	0.264	0.336	0.538	0
0.808	0.031	0.811	0.546	0.838	1.356	0
0.323	0.798	0.025	0.961	1.121	**0.24**	1
0.074	0.962	0.184	0.159	1.035	0.344	0
0.104	0.061	0.189	0.383	0.164	0.572	0
0.931	0.542	0.747	0.905	**0.66**	1.652	1
0.336	0.275	0.051	0.493	0.611	0.544	0

7.2.1. Creating perfect autoencoders

We manually built autoencoders for each linear combination mentioned above in Section 7.1.4, so they can produce an output identical to the input. The autoencoders include an input layer, an inner layer, and output layers. For each autoencoder, we set the weights from the input layer to the inner layer to one for all of the independent features and to zero for the dependent features. The weight from the inner neuron that gets the value of the independent feature X_i to X'_i in the output layer will be one or minus one, depending on the linear combination of the features. If the linear combination is $X_5 = X_1+X_2$, then X_1 and X_2 are the independent features. Therefore, the weights from X_1 and X_2 in the input layer to the matching neurons in the inner layer will be one. The weights from those inner neurons to X'_1 and X'_2



(a) Model 1: $X_1+X_2 = X_5$, $X_3+X_4 = X_6$

(b) Model 2: $X_2 = X_5 - X_1$, $X_4 = X_6 - X_3$

(c) Model 3: $X_1 = X_5-X_2$, $X_3 = X_6-X_4$

Fig. 5. Perfect autoencoders with weights matching the linear combinations (black lines=weight of one, red lines=weight of minus one, gray lines=weight of zero).

will also be one. X'_5 in the output layer will get a weight of one from the neurons that received the value of X_1 and X_2 in the inner layer, to match the linear combination $X_5 = X_1+X_2$.

On the other hand, if the relationship is $X_1 = X_5$-X_2, then X_5 and X_2 are the independent features. Therefore, the weights from X_5 and X_2 in the input layer to the matching neuron in the inner layer will be one. The weights from those inner neurons to X'_5 and X'_2 will be one. X'_1 in the output layer will receive a weight of one from the neuron that received the value of X_5 in the inner layer, and a weight of minus one from the neuron that received the value of X_2 in the inner layer, to match the linear combination $X_1 = X_5$-X_2. Some examples for the perfect autoencoders are presented in Figs. 5(a), 5(b), and 5(c).

Results. Since we know the dependencies of the different features of this artificial dataset, we expect that our explanation model will provide the correct explanation for an anomaly. For example, if X_1 is 0.35 and X_2 is 0.24, then X_5 should be equal to 0.59, but if this record is identified as an anomaly, X_5 will randomly receive any value between zero and one. In the autoencoder that represents the linear combination $X_1 + X_2 = X_5$, we expect that X_5 will have a high reconstruction error, and as a result, this record will be considered an anomaly. We also expect that the explanation model based on SHAP will provide an explanation for the anomaly that includes only features X_1 and X_2 (they will obtain high SHAP values), meaning that these two features explain the reconstruction error of X_5. In the autoencoder that represents the linear combination $X_1 = X_5 - X_2$, we expect that X_1 will have a high reconstruction error, and the explaining features for the anomaly will be X_5 and X_2. Ultimately, we are interested in the set of explanatory features, which are the combination of the features with a high reconstruction error and the features that explain the reconstruction error (the features with the highest SHAP values that contribute or offset the anomaly). The set of explanations we expect to see for an anomaly caused by changing the value of X_5 should be equal to all three linear combinations, as seen in Table 2. We ran our explanation method on 5,000 anomalies, and for each anomaly, we examined whether the set of explanatory features matched our expectations, for all three models. Only seven anomalies were not explained well (one feature was missing), and after investigating these results, we realized that the background set chosen for calculating the SHAP values was responsible for the mismatch. When we increased the size of the background set, all anomalies were explained exactly as expected. This points out the need to examine how to optimally choose the background set, which we plan to do in future research.

Table 2
Features with high reconstruction errors, features that explain the high reconstruction error (have high SHAP values), and the set of explanatory features. This table represents an example of the set of features explaining an anomaly caused by the change of X_5 (without the second linear combination).

Model	High reconstruction error features	Explanatory features	Set of features explaining the anomaly
Model 1	X_5	X_1, X_2	X_1, X_2, X_5
Model 2	X_2	X_5, X_1	X_1, X_2, X_5
Model 3	X_1	X_5, X_2	X_1, X_2, X_5

7.2.2. Creating more complex perfect autoencoders

We also tried to use our perfect autoencoder method on binary data; to do this, we generated 1,000,000 records with four independent features of random binary values. Next, we created two dependent features that are the output of logical "AND" and "OR" operations of the independent binary features. The first dependent feature X_5 is the result of an "AND" operation between X_1 and X_2. The second dependent feature X_6 is the result of an "OR" operation between X_3 and X_4. Eventually we obtained four independent binary features and two dependent binary features. In order to add complexity to the model, we added 16 independent binary features, so we have a total of 22 binary features. Then, we added noise to 15,000 random records by changing the output of one of the dependent binary features X_5 or X_6.

In this experiment, instead of manually setting the weight to zeros and ones as we did in the previous experiment on the perfect autoencoder, we allowed the autoencoder to train its weights on the binary data generated.

Results. In all of the anomalies we created, the set of explanatory features matched our expectations, meaning it was the correct set of explanatory features.

7.3. Robustness of the explanation model

We developed a procedure to evaluate the robustness of the explanation model, inspired by the benchmark interpretability method (BIM) presented recently by Yang and Kim (2019). To construct this benchmark, they added noise objects to images and trained a classifier, expecting that the noise objects would not play a role in explaining the classification. To evaluate our model's robustness, we created noise features within the dataset. Then, we observed the use of these features in the explanations of anomalies when our suggested method is used, as explained earlier, with SHAP and LIME (Ribeiro et al., 2016). In this aspect of our evaluation, we aim to show that the explanations obtained when using SHAP are more robust to noise than those obtained when using LIME.

7.3.1. Creating the explanation feature set

Similar to the way we examined the "perfect" autoencoder (described in 7.2.1), we need to define the $ExplanatoryFeatureSet$ for the robustness evaluation. To create the $ExplanatoryFeatureSet$, we first need to define two parameters: $ReconstructionErrorPercent$ and $SHAPvaluesSelection$.

$ReconstructionErrorPercent$. This parameter is the percent of the total reconstruction error we want to explain. When we visually depict the output of our method, we chose to show 80% of the reconstruction error. This value was obtained through trial and error on the Warranty Claims dataset. This value affects the number of features we explain, and therefore it affects the number of features we receive in the final $ExplanatoryFeatureSet$ for a specific record. In addition, this parameter affects the runtime of our method; as its value increases, we will explain a larger portion of the reconstruction error, i.e., calculate SHAP values to explain the reconstruction error of more features.

$SHAPvaluesSelection$. This parameter represents the method for selecting the features used to explain the feature with the reconstruction error (i.e., the anomaly that needs to be explained). The selected features are added to $ExplanatoryFeatureSet$. We used the following selection methods:

1. Selecting features with a SHAP value higher than the mean SHAP value,
2. Selecting features with a SHAP value higher than the median SHAP value, and
3. Selecting the five features with the highest SHAP values.

Table 3
Reconstruction error of features.

Feature name	Reconstruction error	Cumulative error
F_1	0.95	0.95
F_2	0.83	1.78
F_3	0.52	2.3
F_4	0.43	2.73
F_5	0.24	2.97

The combination of $ReconstructionErrorPercent$ and $SHAPvaluesSelection$ creates the $ExplanatoryFeatureSet$ in the following way: The features whose reconstruction errors sum to $ReconstructionErrorPercent$ of the total reconstruction error of the instance are selected. Each such feature is added to the $ExplanatoryFeatureSet$, along with the set of features that explain it that meet the $SHAPvaluesSelection$ criteria. The order of the $ExplanatoryFeatureSet$ is set so that each explained feature that appears is followed by the selected features that explain it.

In this process of creating the $ExplanatoryFeatureSet$ we may encounter features that already appear on the list. For example, one explained feature might explain another feature's reconstruction error. In that case, a feature will appear in the $ExplanatoryFeatureSet$ only once.

Example. We received the reconstruction errors in Table 3 for a given record with a total reconstruction error of 2.97. Let us assume that $ReconstructionErrorPercent$=0.5; in this case, we explain only the first two features, F_1, F_2, because 50% of the total reconstruction error is 1.485. Next, we use SHAP to explain the reconstruction errors of F_1 and F_2. The explanations for the reconstruction errors of F_1 and F_2 are shown in Table 4. We demonstrate the creation of $ExplanatoryFeatureSet$ for each option of $SHAPvaluesSelection$.

1. **SHAP values higher than the mean SHAP value.** The mean SHAP value of the four explanatory features we obtain in the explanation of F_1 is 0.4. We can see that only the first two features have a SHAP value greater than 0.4, so we add only F_2 and F_5 to $ExplanatoryFeatureSet$, after the explained feature F_1. The mean SHAP value of the five explanatory features we obtain in the explanation of F_2 is 0.268; since only the first feature F_3 has a SHAP value greater than 0.285, it is added to $ExplanatoryFeatureSet$ (after the explained feature F_2). The final $ExplanatoryFeatureSet$ after removing duplicates is $[F_1, F_2, F_5, F_3]$ (the order is important for calculating the robustness measure).
2. **SHAP values higher than the median SHAP value.** In this example, the median equals the mean of the four explanatory features we obtain in the explanation of F_1 is 0.4., so we only add F_2 and F_5 to $ExplanatoryFeatureSet$, after the explained feature F_1. The median SHAP value of the five explanatory features we obtain in the explanation of F_2 is 0.2, so the first two features, F_3 and F_6, have a SHAP value greater than 0.2 and thus are added to $ExplanatoryFeatureSet$ (after the explained feature F_2). The final $ExplanatoryFeatureSet$ after removing duplicates is $[F_1, F_2, F_5, F_3, F_6]$.
3. **Top five SHAP values.** In this example we have only four explaining features for F_1 and five explaining features for F_2, so we will add all of them to $ExplanatoryFeatureSet$ in the order they appear in the table: first we add F_1, then F_2, F_5, F_3, F_4. The next feature we want to explain F_2 is already in the set, so we add the features that explain F_2 and are not yet in the set to the $ExplanatoryFeatureSet$. The final set is $[F_1, F_2, F_5, F_3, F_4, F_6]$.

The same process of creating $ExplanatoryFeatureSet$ is performed when examining the explanations of our suggested method using LIME instead of SHAP.

Table 4
Features that explain the reconstruction errors of F_1 and F_2.

Explained feature	Explanatory features	SHAP value
F_1	F_2	0.7
	F_5	0.5
	F_3	0.3
	F_4	0.1
F_2	F_3	0.7
	F_6	0.25
	F_4	0.2
	F_1	0.16
	F_5	0.03

7.3.2. Robustness test process

We randomly chose features to become noise features. The values of each such feature were changed randomly to a value within the original range of the feature taken from a uniform distribution. Then, we built an autoencoder using the normal (not anomalous) instances. After training the autoencoder, we explained the anomalies from the dataset using our explanation method with Kernel SHAP. We then used our proposed method but with LIME instead of SHAP and compared the two explanatory feature sets. We expect to see as few explanations that include the noise feature as possible. If the noise feature exists in the $ExplanatoryFeatureSet$, we expect it to be located at a low position, meaning that it is not very important in the explanation. Since we changed the noise features to a random value, we performed this process five times for each of the explanation methods (SHAP and LIME) for each of the noise features. It is important to note that in each iteration $k = 1..5$, we examine the explanations provided by SHAP and LIME on the same data.

7.3.3. Sensitivity analysis

To evaluate our method, we performed sensitivity analysis. We explained each anomaly using our method with $ReconstructionErrorPercent$ = [0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9], and for each value of $ReconstructionErrorPercent$ we used all of the possible $SHAPvaluesSelection$ methods. Each combination of the values of $ReconstructionErrorPercent$ and $SHAPvaluesSelection$ (mean, median, and top five) created a different $ExplanatoryFeatureSet$.

7.3.4. Evaluation criterion

We measure the performance of this evaluation method using the mean reciprocal rank (MRR) measure, which considers the position of the noise feature in the explanatory feature set; the higher the noise feature appears in the explanatory feature set, the higher the value of the MRR. This means that we aim for as low a MRR value as possible. The order of operations in the process are as follows:

1. Choose a random feature (the noise feature),
2. Change its values to random values that distribute uniformly,
3. Train the autoencoder model on the data that contains the noise feature with new random values, and
4. Explain anomalous instances with the suggested method, once using Kernel SHAP and once with LIME, and calculate the mean MRR for each of them.

Results. For each noise feature and $ExplanatoryFeatureSet$ (with its two parameters, $ReconstructionErrorPercent$ and $SHAPvaluesSelection$), we calculated the mean MRR of SHAP and LIME, and then we used a paired t-test to determine their significance. Figs. 6 and 7 show the graphs of the MRR for the different explanation sets, based on the $ReconstructionErrorPercent$ and $SHAPvaluesSelection$ values, for the two datasets. The mean MRR of SHAP is significantly lower (p-value < 0.001) than the mean MRR of LIME for all of the explanation sets on both datasets. This means that LIME tends to use the noise features for explanations more, so it is less robust than SHAP.

7.4. Effectiveness of explanations in reducing anomaly score

In the literature, there are a few works (Lundberg et al., 2018; Lundberg & Lee, 2017; Samek et al., 2017; Shrikumar et al., 2017) that evaluate prediction explanations by changing the features that most explain the predicted class, with the goal of changing the predicted class. We evaluate our method using the same idea but adapt it to anomalies and an unsupervised model. The autoencoder provides an anomaly score for an instance, which is the sum of the reconstruction errors of all of the features. Our method explains the anomaly using the features with the highest reconstruction error and the features that explain those features. In this evaluation, we change the values of the first feature in $ExplanatoryFeatureSet$ and the features that explain the first feature. We also chose to evaluate the method in this way given the way the experts used the explanations in the experts' evaluation. They examined the anomaly using the explanations, starting with the feature with the highest reconstruction error and its explanatory features, and then moved on to the next feature. This means that they expect that the features at the beginning of the $ExplanatoryFeatureSet$ will help them the most in understanding the cause of the anomaly. The $ExplanatoryFeatureSet$ is built in two ways:

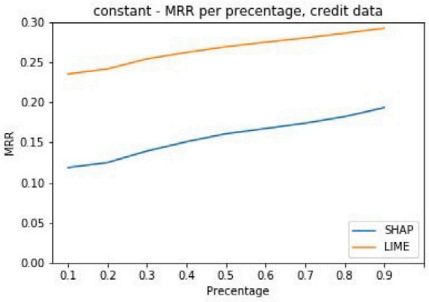
1. Explain anomalous instances with our method using Kernel SHAP, and
2. Explain anomalous instances with our method using LIME.

We changed the values of the features that exist in the $ExplanatoryFeatureSet$, expecting that this change would decrease the total reconstruction error, measured by mean squared error. To examine the impact of changing the anomaly feature values on the reconstruction error, we need to examine different ways of changing these values:

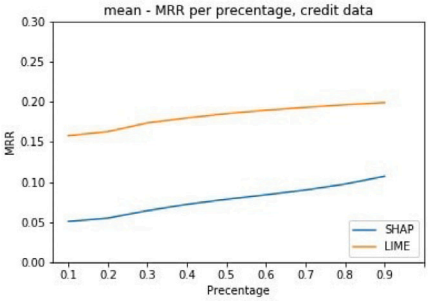
1. Replacing values with mean values — we calculate the mean values of each of the features in the dataset and change the feature value to its mean value.
2. Replacing a value with its predicted (reconstructed) value by the trained autoencoder.

Results. Table 5 presents (for two datasets) the average MSE (anomaly score) of 400 anomalies before changes, the average MSE after changing the value of the feature with the highest reconstruction error (to a mean value and a predicted value), and the MSE after changing the values of the explanatory features of the feature that needs to be explained, with both explanation sets (SHAP and LIME); for each method, the value was changed to a mean value and a predicted value. We checked for statistical significance using a one-way ANOVA with repeated measures, with p-value < 0.05, followed by a paired t-test, with confidence level p-value < 0.05, between the methods ($ExplanatoryFeatureSet$ built using SHAP and LIME) and the different value change options (mean, predicted). All tests were performed with vectors of 400 anomaly scores. The tests were done using a few configurations: (1) with the same parameter configuration for all the methods (value changes to a mean value, and another test for all methods with value changes to the predicted value), (2) with the best parameter configuration for each method (lowest average reconstruction error).

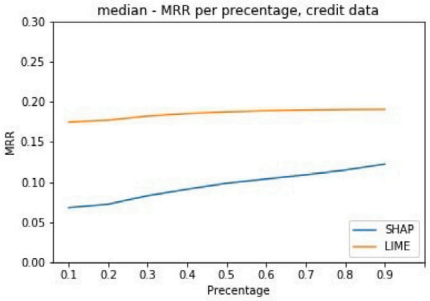
Our results indicate that for both datasets, we were able to significantly reduce the anomaly score when using the explanation set created with SHAP to change the feature values to the mean value or the predicted value more than when using the explanation set created with LIME. The reduction is significant between the anomaly score after changing the feature with highest reconstruction error and the anomaly score after changing values from the explanation set. We can also see in the table that changing the value to the predicted value is better than changing it to the mean value.



(a) Explaining five features with top five SHAP values



(b) Explaining features higher than the mean



(c) Explaining features higher than the median

Fig. 6. Credit Card dataset — MRR of SHAP vs. LIME. The x-axis presents the $ReconstructionErrorPercent$, and the y-axis presents the MRR. Each graph represents a different way of creating the set.

Table 5
Comparison of MSE reduction by changing values of features that explain that anomalies.

Data	Mean MSE	Mean MSE after changing feature with highest error		SHAP		LIME	
		Mean	Predicted	Mean	Predicted	Mean	Predicted
Credit	0.0252	0.0197	0.0197	**0.0173**	**0.0173**	0.0189	0.0187
KDD	0.00053	0.0011	0.00046	**0.0006**	**0.0003**	0.0023	0.00038

7.5. Experts' evaluation

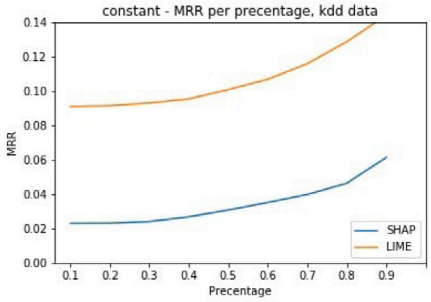
As part of a project aimed at developing an anomaly detection method for monitoring the cost of the warranty claims of a large car manufacturer, we developed an autoencoder-based anomaly detector. The detection of fraud or human error is part of an effective cost-monitoring process which is extremely important to the company, enabling them to reduce costs, improve their products, and better serve their customers. Until now, the domain experts at the company have produced reports based on predefined rules (according to KPIs) to reveal irregularities in warranty claims. The output of the autoencoder-based anomaly detector revealed anomalies that the domain experts were unable to detect using the existing process. However, an explanation of the anomalies was needed in order to convince the domain experts of the correctness of the anomalies found. In order to accomplish this, we used an autoencoder to detect anomalies from 15,000 warranty claims, with 1,000 features. Domain experts received a list of the 114 top anomalies with the visual depiction for explaining the anomalies that we provided. They were instructed to decide whether the anomaly should be inspected further, using their existing system and the visual depiction. We conducted interviews with four of the experts after the experiment, and they reported that the explanations provided a clear indication of how to examine the anomalies (more specifically, they noted that by using the visual depiction, they were able to focus first on the most important explanatory *contributing* features (depicted in a darker color) to examine the anomaly).

We note that in over 85% of the claims, the first one to three explanatory features explained why the claim was anomalous. The domain experts also said that our method helped them handle complicated claims more than it helped them with common, easy claims. It took around six hours for them to adapt to working with the visual depiction tool, but after that, they were comfortable with these visualizations and satisfied with the explanations.

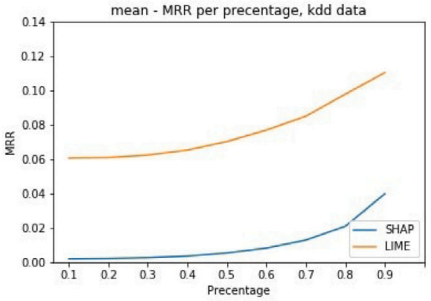
7.6. Method implementation

The method was implemented in Python. The code for the explanation method can be found in GitHub.[1] We used Keras for building the autoencoder, with the following parameters: batch size = 64, ephocs = 1000, early stopping on the validation set loss, optimizer = adam, and loss = mean squared error. The autoencoder is built with five layers. The parameters were chosen after trial and error using a validation set. The input and output layers of the autoencoder contain n neurons,

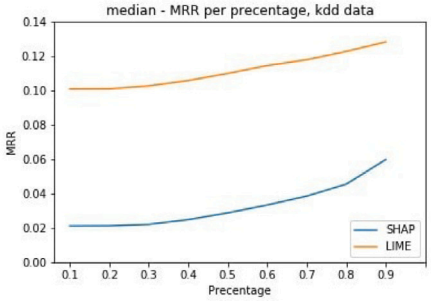
[1] https://github.com/ronniemi/explainAnomaliesUsingSHAP



(a) Explaining five features with top five SHAP values



(b) Explaining the features higher than the mean



(c) Explaining the features higher than the median

Fig. 7. KDD Cup '99 dataset — MRR of SHAP vs. LIME. The x-axis presents the $ReconstructionErrorPercent$, and the y-axis presents the MRR. Each graph represents a different way of creating the set.

Table 6
Autoencoder architecture with the number of parameters for the KDD Cup 1999 dataset (External Data Source, 2018b).

Layer	Output Shape	Number of parameters
Input layer	30	0
dense_1	15	465
dense_2	7	112
dense_3	15	120
dense_4	30	480

where n is the number of features in the dataset. The second and fourth layers contain $n/2$ features, and the inner layer contains $n/4$ features. Table 6 shows the architecture of the autoencoder with the number of parameters for the KDD Cup 1999 dataset (External Data Source, 2018b)

We used kernel explainer from the package SHAP.[2] For the evaluations we also used LIME tabular.[3] For Kernel SHAP we used the default number of samples to choose the coalitions and 200 instances from the dataset as a background for both Kernel SHAP and LIME (the same background instances) in each comparison.

7.7. Computational efficiency

For each instance we want to explain the runtime complexity of our method, which depends on: (1) the size of the background set j; (2) the number of sub-samples used to estimate Shapley values $|s|$; this is the number of coalitions sampled from all of the possible coalitions; (3) the number of features in the dataset n; and (4) the number of features that were not reconstructed correctly that we want to explain $topMfeatures$.

Each time we use Kernel SHAP, a weighted linear regression model is built with a synthetic dataset created from the background set and the coalitions, so the synthetic dataset contains $j * |s|$ instances and n features.

The operations required to build the linear regression are $(X^TX)^{-1}X^Ty$ (Tan et al., 2016). X^TX is done in $O(n^2 * j * |s|)$. By default in Kernel SHAP, $|s|$ is a function of the number of features n, so the complexity of that product is $O(n^2 * j * n)$, which is $O(n^3 * j)$. Inverting the result is done in $O(n^3)$ operations. The runtime complexity of the multiplication and inversion stem from the implementation of the NumPy package on those operations. The rest of the operations are less complex, so the complexity of building the linear regression for estimating the Shapley values is $O(n^3 * j)$. In our method we use Kernel SHAP for each feature we want to explain in $topMfeatures$, which is usually (in all four datasets we used) less than 10 times. Thus, the complexity is $O(n^3 * j)$.

Kernel SHAP's runtime increases with the number of features. When explaining anomalies, a fast computation time is not always the most important consideration, since we can run explanations periodically and provide them to a domain expert for offline investigation, as we did with the domain experts of the car manufacturer project.

Other explanation methods can be used within our method instead of Kernel SHAP if speed is important for real time or other purposes. For example, Deep explainer (Lundberg & Lee, 2017) can be used.

8. Discussion

In this paper, we presented an approach for explaining anomalies identified by an autoencoder using Kernel SHAP. We examined the correctness of the method on an artificial dataset and perfect autoencoders.

[2] https://github.com/slundberg/shap/blob/master/shap/explainers
[3] https://github.com/marcotcr/lime/blob/master/lime

In the process, we realized that in order to evaluate our method in other ways, we needed to create an explanation set. While this set is not as informative to domain experts as the visual depiction we provide, where they can better understand which features were most important in creating a high reconstruction error, it is necessary for evaluating the set of explanatory features; furthermore, the order of the features within the set is important to our evaluation process.

With those things in place, we evaluated the robustness of the explanations and the effectiveness of the explanations in reducing the anomaly score on real-world data using the $ExplanatoryFeatureSet$ that was built using Kernel SHAP, as suggested in this paper, but also using LIME for comparison. We assessed the robustness of the method by changing each time one feature to noise feature and thus should not explain an anomaly. We checked if and where the explanation methods use the noise feature in the explanation set. We found that SHAP uses the noise feature significantly less and at a lower position in the explanation set than LIME. This evaluation was done using different methods of creating the explanation sets. The effectiveness of the explanations in reducing the anomaly score was similarly assessed on both SHAP and LIME. In this evaluation, we changed the first features in the explanation set to other values (mean value, predicted value) and examined the change in the anomaly score for all three methods. The idea behind this evaluation was that if the features in the explanation set explain the anomaly, then when we change them, the instance would be less anomalous, meaning its anomaly score would be reduced. In this evaluation we saw that SHAP reduces the reconstruction error more than LIME.

It is worth mentioning the following insights regarding the suggested method:

1. *The goal of the explanation method:* We presented a method for explaining anomalies using (1) the features with the highest reconstruction errors from an autoencoder, and (2) the features that are most important in affecting the reconstruction error of the feature need to be explained. The main aim of the research was to provide a more comprehensive explanation to the user by focusing on the connection between the features with high reconstruction error and the features that are most important in affecting the reconstruction error.
2. *Background set:* Since this method treats the autoencoder model as a black-box, we need a background set to create a local explanation model. The background set in this research consists of 200 instances from the dataset, without any constraints or considerations. For a fair comparison, the same background set was used with both SHAP and LIME. Choosing an appropriate background set can lead to different explanation models. We are currently addressing this important issue in another study. A possible direction for choosing the background set is to use instance selection methods (Carbonera & Olszewska, 2019; Olvera-López et al., 2010).
3. *Visualization:* In this research, we presented a visual depiction of the explanation as part of our joint work with a large car manufacturer. Further research should be performed to explore other ways of presenting the explanations. We believe that the form the presentation takes will likely depend on the needs of the user, as well as other factors.

9. Conclusion and future work

We developed a method that uses SHAP values, which are based on game theory, to explain anomalies revealed by an autoencoder. The feedback obtained from the domain experts about the explanations for the anomalies was positive. In our evaluation of the explanation method using perfect autoencoders, we showed that the set of the features that have high reconstruction errors, together with the features that contribute or offset the anomaly, are what explains an anomaly. The evaluation of the proposed method's robustness showed that creating a set of explanatory features as part of our explanation method using SHAP is more robust than creating the set using LIME. We plan to further develop the proposed method by examining the background set used for the explanation model and evaluate the explanation method with more complicated autoencoders and additional datasets.

CRediT authorship contribution statement

Liat Antwarg: Conceptualization, Methodology, Software, Investigation, Writing – original Draft, Visualization. **Ronnie Mindlin Miller:** Software, Formal analysis, Data curation. **Bracha Shapira:** Supervision, Writing - review & editing. **Lior Rokach:** Supervision, Writing - review & editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

References

Adadi, A., & Berrada, M. (2018). Peeking inside the black-box: A survey on explainable artificial intelligence (XAI). *IEEE Access*, *6*, 52138–52160.

Aggarwal, C. C. (2015). Outlier analysis. In *Data mining* (pp. 237–263). Springer.

Amarasinghe, K., Kenney, K., & Manic, M. (2018). Toward explainable deep neural network based anomaly detection. In *2018 11th international conference on human system interaction* (pp. 311–317). IEEE.

An, J., & Cho, S. (2015). Variational autoencoder based anomaly detection using reconstruction probability. *Special Lecture on IE*, *2*, 1–18.

Arp, D., Spreitzenbarth, M., Hubner, M., Gascon, H., Rieck, K., & Siemens, C. (2014). DREBIN: Effective and explainable detection of android malware in your pocket. In *Ndss (Vol. 14)* (pp. 23–26).

Arrieta, A. B., Díaz-Rodríguez, N., Del Ser, J., Bennetot, A., Tabik, S., Barbado, A., García, S., Gil-López, S., Molina, D., & Benjamins, R. (2020). Explainable artificial intelligence (XAI): Concepts, taxonomies, opportunities and challenges toward responsible AI. *Information Fusion*, *58*, 82–115.

Ben-Gal, I. (2005). Outlier detection. In *Data mining and knowledge discovery handbook* (pp. 131–146). Springer.

Bengio, Y., & LeCun, Y. (2007). Scaling learning algorithms towards AI. *Large-Scale Kernel Machines*, *34*(5), 1–41.

Bergman, L., & Hoshen, Y. (2020). Classification-based anomaly detection for general data. In *ICLR 2020*.

Bertsimas, D., & Dunn, J. (2017). Optimal classification trees. *Machine Learning*, *106*(7), 1039–1082.

Bertsimas, D., Orfanoudaki, A., & Wiberg, H. (2018). Interpretable clustering via optimal trees. ArXiv preprint arXiv:1812.00539.

Breunig, M. M., Kriegel, H.-P., Ng, R. T., & Sander, J. (2000). LOF: identifying density-based local outliers. In *Proceedings of the 2000 ACM SIGMOD international conference on management of data* (pp. 93–104).

Carbonera, J. L., & Olszewska, J. I. (2019). Local-set based-on instance selection approach for autonomous object modelling. *International Journal of Advanced Computer Science and Applications*, *10*(12), Paper–1.

Chandola, V., Banerjee, A., & Kumar, V. (2009). Anomaly detection: A survey. *ACM Computing Surveys*, *41*(3), 15.

Chen, J., Sathe, S., Aggarwal, C., & Turaga, D. (2017). Outlier detection with autoencoder ensembles. In *Proceedings of the 2017 SIAM international conference on data mining* (pp. 90–98). SIAM.

Collaris, D., Vink, L. M., & van Wijk, J. J. (2018). Instance-level explanations for fraud detection: A case study. ArXiv preprint arXiv:1806.07129.

Doshi-Velez, F., & Kim, B. (2017). A roadmap for a rigorous science of interpretability. *Stat*, *1050*, 28.

Erfani, S. M., Rajasegarar, S., Karunasekera, S., & Leckie, C. (2016). High-dimensional and large-scale anomaly detection using a linear one-class SVM with deep learning. *Pattern Recognition*, *58*, 121–134.

External Data Source (2018a). *Credit card fraud detection*. IMPACT, http://dx.doi.org/10.23721/100/1504318, URL: https://www.impactcybertrust.org/dataset_view?idDataset=1276.

External Data Source (2018b). *KDD cup 1999 data*. IMPACT, http://dx.doi.org/10.23721/100/1478801, URL: https://www.impactcybertrust.org/dataset_view?idDataset=937.

Friedman, J. H. (2001). Greedy function approximation: a gradient boosting machine. *The Annals of Statistics*, 1189–1232.

Gilpin, L. H., Bau, D., Yuan, B. Z., Bajwa, A., Specter, M., & Kagal, L. (2018). Explaining explanations: An overview of interpretability of machine learning. In *2018 IEEE 5th international conference on data science and advanced analytics* (pp. 80–89). IEEE.

Golan, I., & El-Yaniv, R. (2018). Deep anomaly detection using geometric transformations. In *Advances in neural information processing systems* (pp. 9758–9769).

Goodall, J. R., Ragan, E. D., Steed, C. A., Reed, J. W., Richardson, G. D., Huffer, K. M., Bridges, R. A., & Laska, J. A. (2019). Situ: Identifying and explaining suspicious behavior in networks. *IEEE Transactions on Visualization and Computer Graphics*, *25*(1), 204–214.

Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep learning*. MIT press.

Goodman, B., & Flaxman, S. (2017). European Union regulations on algorithmic decision-making and a "right to explanation". *AI Magazine*, *38*(3), 50–57.

Guidotti, R., Monreale, A., Ruggieri, S., Turini, F., Giannotti, F., & Pedreschi, D. (2018). A survey of methods for explaining black box models. *ACM Computing Surveys*, *51*(5), 93.

Gunning, D. (2017). *Explainable artificial intelligence (xai)*. Defense Advanced Research Projects Agency (DARPA), Nd Web.

Hawkins, S., He, H., Williams, G., & Baxter, R. (2002). Outlier detection using replicator neural networks. In *International conference on data warehousing and knowledge discovery* (pp. 170–180). Springer.

Hinton, G. E., Osindero, S., & Teh, Y.-W. (2006). A fast learning algorithm for deep belief nets. *Neural Computation*, *18*(7), 1527–1554.

Hinton, G. E., & Salakhutdinov, R. R. (2006). Reducing the dimensionality of data with neural networks. *Science*, *313*(5786), 504–507.

Hodge, V., & Austin, J. (2004). A survey of outlier detection methodologies. *Artificial Intelligence Review*, *22*(2), 85–126.

Hoffman, R. R., Mueller, S. T., Klein, G., & Litman, J. (2018). Metrics for explainable AI: Challenges and prospects. ArXiv preprint arXiv:1812.04608.

Jolliffe, I. (2011). *Principal component analysis*. Springer.

Kauffmann, J., Müller, K.-R., & Montavon, G. (2020). Towards explaining anomalies: a deep taylor decomposition of one-class models. *Pattern Recognition*, *101*, Article 107198.

Kindermans, P.-J., Hooker, S., Adebayo, J., Alber, M., Schütt, K. T., Dähne, S., Erhan, D., & Kim, B. (2017). The (UN) reliability of saliency methods. *Stat*, *1050*, 2.

Kopp, M., Pevnỳ, T., & Holeňa, M. (2020). Anomaly explanation with random forests. *Expert Systems with Applications*, *149*, Article 113187.

Lipton, Z. C. (2018). The mythos of model interpretability. *Queue*, *16*(3), 31–57.

Liu, H., & Motoda, H. (2013). *Instance selection and construction for data mining, vol. 608*. Springer Science & Business Media.

Liu, N., Shin, D., & Hu, X. (2018). Contextual outlier interpretation. In *Proceedings of the 27th international joint conference on artificial intelligence* (pp. 2461–2467). AAAI Press.

Liu, S., Wang, X., Liu, M., & Zhu, J. (2017). Towards better analysis of machine learning models: A visual analytics perspective. *Visual Informatics*, *1*(1), 48–56.

Liu, B., Xia, Y., & Yu, P. S. (2000). Clustering through decision tree construction. In *Proceedings of the ninth international conference on information and knowledge management* (pp. 20–29). ACM.

Lundberg, S. M., Erion, G. G., & Lee, S.-I. (2018). Consistent individualized feature attribution for tree ensembles. ArXiv preprint arXiv:1802.03888.

Lundberg, S. M., & Lee, S.-I. (2017). A unified approach to interpreting model predictions. In *Advances in Neural Information Processing Systems* (pp. 4765–4774).

Maaten, L. v. d., & Hinton, G. (2008). Visualizing data using t-SNE. *Journal of Machine Learning Research*, *9*(Nov), 2579–2605.

Melis, D. A., & Jaakkola, T. (2018). Towards robust interpretability with self-explaining neural networks. In *Advances in neural information processing systems* (pp. 7786–7795).

Miller, T. (2019). Explanation in artificial intelligence: Insights from the social sciences. *Artificial Intelligence*, *267*, 1–38.

Mittelstadt, B., Russell, C., & Wachter, S. (2019). Explaining explanations in AI. In *Proceedings of the conference on fairness, accountability, and transparency* (pp. 279–288).

Montavon, G., Samek, W., & Müller, K.-R. (2017). Methods for interpreting and understanding deep neural networks. *Digital Signal Processing*.

Nguyen, Q. P., Lim, K. W., Divakaran, D. M., Low, K. H., & Chan, M. C. (2019). GEE: A gradient-based explainable variational autoencoder for network anomaly detection. In *2019 IEEE conference on communications and network security* (pp. 91–99). IEEE.

Olszewska, J. I. (2019). Designing transparent and autonomous intelligent vision systems. In *ICAART (No. 2)* (pp. 850–856).

Olvera-López, J. A., Carrasco-Ochoa, J. A., Martínez-Trinidad, J. F., & Kittler, J. (2010). A review of instance selection methods. *Artificial Intelligence Review*, *34*(2), 133–143.

Palczewska, A., Palczewski, J., Robinson, R. M., & Neagu, D. (2014). Interpreting random forest classification models using a feature contribution method. In *Integration of reusable systems* (pp. 193–218). Springer.

Pang, G., Shen, C., Cao, L., & Hengel, A. v. d. (2020). Deep learning for anomaly detection: A review. ArXiv preprint arXiv:2007.02500.

Paula, E. L., Ladeira, M., Carvalho, R. N., & Marzagão, T. (2016). Deep learning anomaly detection as support fraud investigation in brazilian exports and anti-money laundering. In *Machine learning and applications (ICMLA), 2016 15th IEEE international conference on* (pp. 954–960). IEEE.

Radev, D. R., Jing, H., Styś, M., & Tam, D. (2004). Centroid-based summarization of multiple documents. *Information Processing & Management*, *40*(6), 919–938.

Ribeiro, M. T., Singh, S., & Guestrin, C. (2016). Why should i trust you?: Explaining the predictions of any classifier. In *Proceedings of the 22nd ACM SIGKDD international conference on knowledge discovery and data mining* (pp. 1135–1144). ACM.

Rumelhart, D. E., Hinton, G. E., & Williams, R. J. (1985). *Learning internal representations by error propagation: Technical report*, California Univ San Diego La Jolla Inst for Cognitive Science.

Sakurada, M., & Yairi, T. (2014). Anomaly detection using autoencoders with nonlinear dimensionality reduction. In *Proceedings of the MLSDA 2014 2nd workshop on machine learning for sensory data analysis* (p. 4). ACM.

Samek, W., Binder, A., Montavon, G., Lapuschkin, S., & Müller, K.-R. (2017). Evaluating the visualization of what a deep neural network has learned. *IEEE Transactions on Neural Networks and Learning Systems*, *28*(11), 2660–2673.

Shortliffe, E. H., & Buchanan, B. G. (1975). A model of inexact reasoning in medicine. *Mathematical Biosciences*, *23*(3–4), 351–379.

Shrikumar, A., Greenside, P., & Kundaje, A. (2017). Learning important features through propagating activation differences. In *Proceedings of the 34th international conference on machine learning-volume 70* (pp. 3145–3153). JMLR. org.

Singh, K., & Upadhyaya, S. (2012). Outlier detection: applications and techniques. *International Journal of Computer Science Issues (IJCSI)*, *9*(1), 307.

Song, H., Jiang, Z., Men, A., & Yang, B. (2017). A hybrid semi-supervised anomaly detection model for high-dimensional data. *Computational Intelligence and Neuroscience*, *2017*.

Štrumbelj, E., Kononenko, I., & Šikonja, M. R. (2009). Explaining instance classifications with interactions of subsets of feature values. *Data & Knowledge Engineering*, *68*(10), 886–904.

Takeishi, N. (2019). Shapley Values of reconstruction errors of PCA for explaining anomaly detection. In *2019 international conference on data mining workshops* (pp. 793–798). IEEE.

Takeishi, N., & Kawahara, Y. (2020). On anomaly interpretation via Shapley values. ArXiv preprint arXiv:2004.04464.

Tan, P.-N., Steinbach, M., & Kumar, V. (2016). *Introduction to data mining*. Pearson Education India.

Yang, M., & Kim, B. (2019). BIM: Towards quantitative evaluation of interpretability methods with ground truth. ArXiv preprint arXiv:1907.09701.

Zhou, C., & Paffenroth, R. C. (2017). Anomaly detection with robust deep autoencoders. In *Proceedings of the 23rd ACM SIGKDD international conference on knowledge discovery and data mining* (pp. 665–674).